

ALF basics: Lattices

Jonas Schwab

ALF workshop Würzburg

July 15-19, 2024

Outline

Lattice Components

- Bravais Lattice

- Unit cell

- Keeping track of the orbital and unit cell structure

Predefined Lattices

- Honeycomb lattice

- Open boundary conditions: N-leg ladder

- All predefined lattices

Lattice: Bravais Lattice + Unit cell

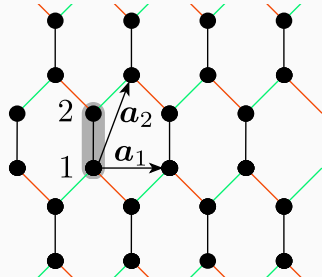
Example: Honeycomb

- Bravais Lattice: Triangular lattice:

$$\mathbf{a}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \mathbf{a}_2 = \begin{pmatrix} \frac{1}{2} \\ \frac{\sqrt{3}}{2} \end{pmatrix}$$

- Two orbitals per unit cell:

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 0 \\ \frac{1}{\sqrt{3}} \end{pmatrix}$$



Bravais Lattice: FORTRAN *type Lattice*

- Finite 2d Bravais Lattice with periodic boundary conditions
- Defined by four 2d vectors $\mathbf{a}_1, \mathbf{a}_2, \mathbf{L}_1, \mathbf{L}_2$
- $\mathbf{a}_1, \mathbf{a}_2$: Span real space $\mathbf{R} = i_1 \mathbf{a}_1 + i_2 \mathbf{a}_2$
- $\mathbf{L}_1, \mathbf{L}_2$: Define periodic boundary conditions $\hat{c}_{\mathbf{R}+\mathbf{L}_i} = \hat{c}_{\mathbf{R}}$
⇒ Tilted lattices possible
- For one-dimensional system, set $\mathbf{L}_2 = \mathbf{a}_2$

Created through:

```
call Make_Lattice(L1, L2, a1, a2, Latt)
```

Extension to 3d is being implemented

Bravais Lattice: k-space quantization

- Brillouin zone defined by reciprocal lattice vectors $\mathbf{g}_i$, with

$$\mathbf{a}_i \cdot \mathbf{g}_j = 2\pi\delta_{ij}$$

- k-space quantization:

$$\begin{aligned} \mathbf{k} \cdot \mathbf{L}_1 &= k_1 \in \mathbb{Z} \\ \mathbf{k} \cdot \mathbf{L}_2 &= k_2 \in \mathbb{Z} \end{aligned} \quad \Rightarrow \quad \mathbf{k} = k_1 \mathbf{b}_1 + k_2 \mathbf{b}_2$$

$$\begin{aligned} \mathbf{b}_1 &= \frac{2\pi}{(\mathbf{L}_1 \cdot \mathbf{g}_1)(\mathbf{L}_2 \cdot \mathbf{g}_2) - (\mathbf{L}_1 \cdot \mathbf{g}_2)(\mathbf{L}_2 \cdot \mathbf{g}_1)} [(\mathbf{L}_2 \cdot \mathbf{g}_2)\mathbf{g}_1 - (\mathbf{L}_2 \cdot \mathbf{g}_1)\mathbf{g}_2] \\ \mathbf{b}_2 &= \frac{2\pi}{(\mathbf{L}_1 \cdot \mathbf{g}_1)(\mathbf{L}_2 \cdot \mathbf{g}_2) - (\mathbf{L}_1 \cdot \mathbf{g}_2)(\mathbf{L}_2 \cdot \mathbf{g}_1)} [(\mathbf{L}_1 \cdot \mathbf{g}_1)\mathbf{g}_2 - (\mathbf{L}_1 \cdot \mathbf{g}_2)\mathbf{g}_1] \end{aligned}$$

Bravais Lattice: Content of `type Lattice`

Variable	Type	Description
<code>a1_p</code> , <code>a2_p</code>	Real	Unit vectors $\mathbf{a}_1$, $\mathbf{a}_2$
<code>L1_p</code> , <code>L2_p</code>	Real	Vectors $\mathbf{L}_1$, $\mathbf{L}_2$
<code>N</code>	Integer	Number of lattice points $N_{\text{unit-cell}}$
<code>list</code>	Integer	$\mathbf{R}_i = \text{list}(i,1)\mathbf{a}_1 + \text{list}(i,2)\mathbf{a}_2 \equiv i_1\mathbf{a}_1 + i_2\mathbf{a}_2$ $i \in [1, N_{\text{unit-cell}}]$
<code>invlist</code>	Integer	Lattice point from position: $\text{Invlist}(i_1, i_2) = i$.
<code>nnlist</code>	Integer	Nearest neighbor indices: $j = \text{nnlist}(i, n_1, n_2)$ $n_1, n_2 \in [-1, 1]$, $\mathbf{R}_j = \mathbf{R}_i + n_1\mathbf{a}_1 + n_2\mathbf{a}_2$.
<code>imj</code>	Integer	$\mathbf{R}_{\text{imj}(i,j)} = \mathbf{R}_i - \mathbf{R}_j$, with $\text{imj}, i, j \in 1, \dots, N_{\text{unit-cell}}$
<code>BZ1_p</code> , <code>BZ2_p</code>	Real	Reciprocal space vectors $\mathbf{g}_i$
<code>b1_p</code> , <code>b1_p</code>	Real	k -quantization
<code>listk</code>	Integer	$\mathbf{k}_k = \text{listk}(k,1)\mathbf{b}_1 + \text{listk}(k,2)\mathbf{b}_2 \equiv k_1\mathbf{b}_1 + k_2\mathbf{b}_2$
<code>invlistk</code>	Integer	$\text{Invlistk}(k_1, k_2) = k$

Bravais Lattice: Fourier transformation

- Subroutine `Fourier_R_to_K(Xin, Xout, Latt)`

$$Xout(k, :, :, :) = \frac{1}{N_{\text{unit-cell}}} \sum_j e^{-ik_k \cdot R_j} Xin(j, :, :, :)$$

- Subroutine `Fourier_K_to_R(Xin, Xout, Latt)`

$$Xout(j, :, :, :) = \frac{1}{N_{\text{unit-cell}}} \sum_k e^{ik_k \cdot R_j} Xin(k, :, :, :)$$

FORTRAN **type** Unit_cell

Variable	Type	Description
Norb	Integer	Number of orbitals.
N_coord	Integer	Coordination number.
Orb_pos(1..Norb,2[3])	Real	Positions of the orbitals.

⇒ Extended zone scheme

Connecting unit cell and Fermion orbital to Fermion superindex

Total number of orbitals given by:

$$N_{\text{dim}} = \text{Latt}\%N * \text{Latt_Unit}\%\text{Norb}$$

Convention to keep track of the orbital and unit cell structure:

List Fermion superindex $\rightarrow \{\text{Unit cell}\#, \text{orbital}\#\}$

Invlist $\{\text{Unit cell}\#, \text{orbital}\#\} \rightarrow \text{Fermion superindex}$

Do not confuse with `Latt%List`, `Latt%Invlist`

```
allocate(List(Ndim,2), Invlist(Latt%N, Latt_Unit%Norb))
```

```
nc = 0
```

```
do i = 1, Latt%N
```

```
  do no = 1, Latt_Unit%Norb
```

```
    nc = nc + 1
```

```
    List(nc,1) = i
```

```
    List(nc,2) = no
```

```
    Invlist(i,no) = nc
```

```
  enddo
```

```
enddo
```

Predefined Lattices

Subroutine defining prepared list of common lattices:

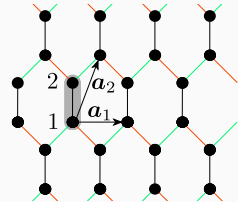
- Square
- Honeycomb
- N_leg_ladder
- Bilayer_square
- Bilayer_honeycomb

Argument	Type	Role	Description
Lattice_type	String	Input	Lattice configuration
L1, L2	Integer	Input	Lattice sizes (set L2=1 for 1D lattices)
Ndim	Integer	Output	Total number of orbitals
List	Integer	Output	Fermion superindex $\rightarrow \{\text{Unit cell\#}, \text{orbital\#}\}$
Invlist	Integer	Output	$\{\text{Unit cell\#}, \text{orbital\#}\} \rightarrow \text{Fermion superindex}$
Latt	Lattice	Output	Sets the lattice
Latt_Unit	Unit_cell	Output	Sets the unit cell

Honeycomb lattice

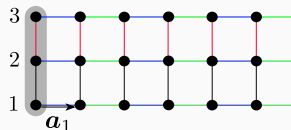
```
a1_p = [1, 0]
a2_p = [0.5, sqrt(3)/2]
L1_p = dble(L1) * a1_p
L2_p = dble(L2) * a2_p
call Make_Lattice(L1_p, L2_p, a1_p, a2_p, Latt)
```

```
Latt_Unit%Norb = 2
Latt_Unit%N_coord = 3
allocate(Latt_Unit%Orb_pos_p(Latt_Unit%Norb, 2))
Latt_Unit%Orb_pos_p(1,:) = [0, 0]
Latt_Unit%Orb_pos_p(2,:) = [0, 1/sqrt(3)]
```



Open boundary conditions: N-leg ladder

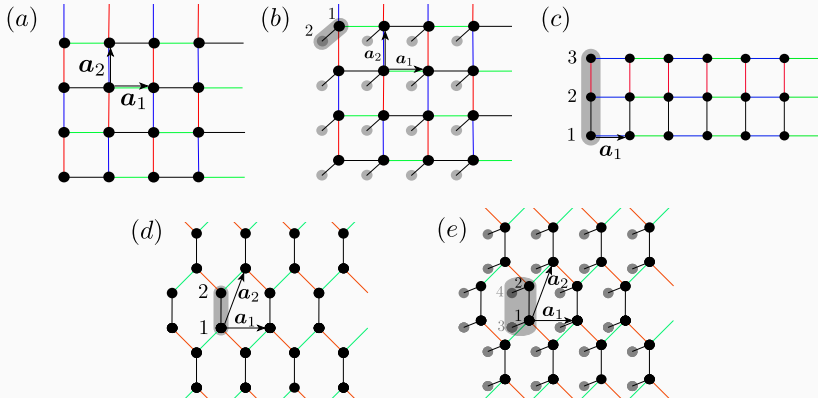
- "Square" with open boundaries in y
- One-dimensional with Orbital size L_2



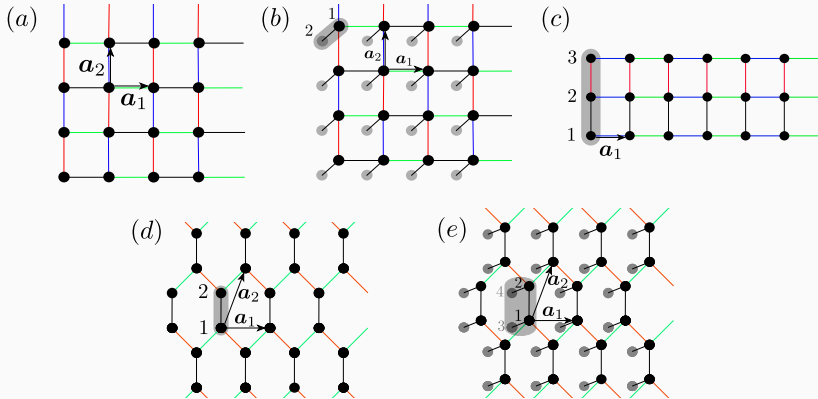
```
a1_p = [1.d0, 0.d0]
a2_p = [0.d0, 1.d0]
L1_p = dble(L1)*a1_p
L2_p = a2_p
call Make_Lattice(L1_p, L2_p, a1_p, a2_p, Latt)
```

```
Latt_Unit%Norb = L2
Latt_Unit%N_coord = 1
allocate(Latt_Unit%Orb_pos_p(Latt_Unit%Norb, 2))
do no = 1, L2
    Latt_Unit%Orb_pos_p(no,1) = 0.d0
    Latt_Unit%Orb_pos_p(no,2) = dble(no-1)
enddo
```

All predefined lattices



All predefined lattices



Thank you for your attention!